

Succinct Dynamic Dictionaries and Trees*

Rajeev Raman¹ and Satti Srinivasa Rao²

¹ Dept. of CS, Univ. of Leicester, Leicester LE1 7RH, UK. r.raman@mcs.le.ac.uk.

² School of CS, Univ. of Waterloo, Canada N2L 3G1. ssrao@monod.uwaterloo.ca

Abstract. We consider space-efficient solutions to two dynamic data structuring problems. We first give a representation of a set $S \subseteq U = \{0, \dots, m-1\}$, $|S| = n$ that supports membership queries in $O(1)$ worst case time and insertions into/deletions from S in $O(1)$ expected amortised time. The representation uses $\mathcal{B} + o(\mathcal{B})$ bits, where $\mathcal{B} = \lceil \lg \binom{m}{n} \rceil$ is the information-theoretic minimum space to represent S . This improves upon the $O(\mathcal{B})$ -bit solutions of Brodnik and Munro [2] and Pagh [16], and uses up to a log-factor less space than search trees or hash tables. The representation can also associate *satellite* data with elements of S .

We also show that a binary tree on n nodes, where each node has $b = O(\lg n)$ -bit data stored at it, can be maintained under node insertions while supporting navigation in $O(1)$ time and updates in $O((\lg \lg n)^{1+\epsilon})$ amortised time, for any constant $\epsilon > 0$. The space used is within $o(n)$ bits of the information-theoretic minimum. This improves upon the equally space-efficient structure of Munro et al. [15], in which updates take $O(\lg^c n)$ time, for some $c \geq 1$.

1 Introduction

Computer science deals with the storage and processing of information. With the rapid proliferation of information, it is increasingly important to focus on the storage requirements of this information, especially as it may be transmitted and copied several times over. Recently there has been a renewal of interest in the study of *succinct* representations of data [1,2,9,13,14,15,18,19], whose space usage is close to the information-theoretic lower bound, but which support operations as efficiently as their usual counterparts. Succinct representations have been found for dictionary operations on static sets, trees, tries, and bounded-genus graphs. With few exceptions [15,19] succinct representations are not dynamic.

In this paper, we consider representing *dynamic dictionaries* succinctly. A dictionary is arguably the single most important abstract data type, a basic formulation of which is as follows. Given a set $S \subseteq U = [m]^1$ support the operations **member**(x, S), which determines whether $x \in S$, and **insert**(x, S), which adds x to S . In many applications, one stores *satellite* data with each element of S , which is retrieved by a successful **member** query. Letting $|S| = n$, perfect hash tables take $O(1)$ worst-case and amortised expected time respectively, for

* Supported in part by UISTRF project 2001.04/IT and EPSRC GR L/92150.

¹ For non-negative integers i , $[i] = \{0, 1, \dots, i-1\}$.

member and insert [5] and balanced trees take $\Theta(\lg n)$ worst-case time for both operations.

The information-theoretic lower bound on the space needed to represent S is $\mathcal{B}(m, n) = \lceil \lg \binom{m}{n} \rceil = n \lg m - n \lg n + O(n)$ (in what follows we abbreviate $\mathcal{B}(m, n)$ by $\mathcal{B}$). Unfortunately, standard solutions use significantly more than $\mathcal{B}$ bits. For example, a solution based on balanced search trees uses at least $n(\lceil \lg m \rceil + \lceil \lg n \rceil)$ bits (one key and one pointer field per node), which can be $\Theta(\log n)$ times more than necessary. A similar situation occurs for other standard solutions, e.g., ‘Cuckoo’ hashing [17] requires $(2 + \epsilon)n \lg m$ bits of storage².

Succinct dictionaries were studied by Cleary [3], who showed how to achieve $(1 + \epsilon)\mathcal{B} + O(n)$ bits with $O(1)$ *expected* time for **member** and **insert** using the strong assumption of *simple uniform hashing* [4, pp. 224]. Brodnik and Munro [2] and Pagh [16] gave optimal $(\mathcal{B} + o(\mathcal{B}))$ -bit representations for static dictionaries, and noted that their solutions may be dynamised, by increasing the space usage to $O(\mathcal{B})$. Obtaining an optimum-space dynamic representation was stated as an open problem by [2], which we solve in this paper. Our solution takes $\mathcal{B} + o(\mathcal{B})$ bits of storage and supports **member** in $O(1)$ worst-case time; **insert** takes $O(1)$ amortised expected time. When s -bit satellite data are associated with each $x \in S$, the space usage becomes $\mathcal{B} + ns + o(\mathcal{B} + ns)$ bits. The *load factor* of a dictionary is the ratio of the number of keys currently in the dictionary to the capacity of the table. Conventional wisdom holds that the time performance of hashing degrades at load factors exceeding $1 - \Omega(1)$. Our result shows that this is wrong in a very real sense.

The model of memory allocation is very important in succinct dynamic data structures. Our result is robust across two models, of theoretical and practical relevance. Earlier dynamic succinct data structures [1,15,19] assumed the existence of a ‘system’ memory manager that would allocate and free memory in variable-sized chunks (see model $\mathcal{M}_A$ below). This approach does not charge the data structure for space wastage due to *external fragmentation* [20, Ch 9]. It is known that if N is the maximum total size of the chunks in use simultaneously, the memory manager needs $\Omega(N \lg N)$ words of memory to serve the requests in the worst case. A decision version of the corresponding offline problem is known to be NP-complete, even when all block sizes are 1 or 2. Hence we also analyse the space requirement under the standard way of measuring space in the RAM model (see model $\mathcal{M}_B$ below).

We also consider the problem of representing dynamic n -node binary trees. The nodes of a binary tree have slots for left and right children; slots that do not point to other tree nodes are said to point to *external* nodes, of which there are $n + 1$. The user may associate $b = O(w)$ -bit *satellite* data with internal nodes, or external nodes, or both. The operations we would like to support, in $O(1)$ time, on the tree are: given a node, finding its parent, left child and right child, and in the course of traversing the tree, the size of the subtree rooted at the current node, the satellite datum associated with the current node (if any) and its pre-order number. We assume, as do previous authors [15], that all traversals

² Recently, Fotakis et al. [6] have improved this to $(1 + \epsilon)n \lg m$

start from the root and end at the root as well. The updates we consider are: adding or deleting a leaf, or inserting a node along an edge.

Unlike a pointer-based representation that uses $\Theta(n \lg n)$ bits, we are interested in representations that come close to the information-theoretic lower bound of $\lg \binom{2n}{n} = 2n - O(\lg n)$ plus the storage for satellite data. Our representation uses $(b+2)n + o(n)$ or $(2b+2)n + o(n)$ bits, depending on whether data is associated only with internal nodes, external nodes, or both. It supports navigation in $O(1)$ time and updates (insertion/deletion of leaf or internal nodes) in $O((\lg \lg n)^{1+\epsilon})$ amortised time, for any constant $\epsilon > 0$. This improves upon the equally space-efficient structure of Munro et al. [15], in which updates take $O(\lg^c n)$ time, for some $c \geq 1$. Munro and Raman [14] earlier showed how to represent a binary tree using $2n + o(n)$ bits and support operations in $O(1)$ time, but this representation is static.

For simplicity, we assume in the dictionary problem that m is a power of 2. We assume the standard *word RAM* model [10] with a word size $w = \lg m$ or $w = \Theta(\lg n)$ for the dictionary and tree problems, respectively. Our dynamic dictionary is based on the static one of Brodnik and Munro [2]. We use their high-level approach, considering several cases depending upon the relative values of n and m , using bucketing and applying table lookup for small universes. We overcome the main obstacle mentioned by [2], namely, how to use dynamic perfect hashing in our data structure while absorbing its high space cost. The dynamic binary tree has a high-level similarity to that of Munro et al. [15].

The rest of the paper is organised as follows: in Section 2 we describe our memory models and resolve management issues. In Section 3 we describe our dynamic dictionary, and Section 4 deals with dynamic trees.

2 Preliminaries

Memory Models. We consider two memory models denoted by $\mathcal{M}_A$ and $\mathcal{M}_B$. In $\mathcal{M}_A$ the algorithm calls built-in “system” procedures *allocate* and *free*. The call *allocate*(k) for some integer $k \geq 0$ returns a pointer p to a block of 2^k consecutive memory locations all initialised to 0. Each memory location is w bits long, as is the pointer p . The call requires $O(2^k)$ time and increases the space usage of the algorithm by $w \cdot 2^k$ bits. The call *free*(p) frees the block of consecutive locations that p was pointing to and reduces the space usage appropriately.

In $\mathcal{M}_B$, the algorithm has access to words numbered $0, \dots, 2^w - 1$. The space usage at any given time is simply $s + 1$ where s is the highest-numbered word currently in use by the algorithm (see [12] for details).

Collections of extendible arrays. An *extendible array* (EA) maintains a sequence of n equal-sized records, each assigned a unique index between 0 and $n - 1$, under the following operations:

- **access**(i): access the record with index i (for reading or writing),
- **grow**: increment n , creating a new record with index n , and
- **shrink**: decrement n , discarding the record with index n .

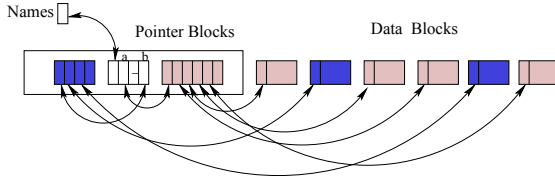


Fig. 1. Maintaining a collection of EAs in $\mathcal{M}_B$

We say that an EA with n records of r bits each has *nominal size* nr bits, and the nominal size of a collection of EAs is the sum of the nominal sizes of the EAs in the collection. We consider the problem of maintaining a collection of EAs under the following operations (the name of an EA is a w -bit integer):

- **create**(r): create an new empty EA with record size r and return its name,
- **destroy**(A): free the array A , and
- **grow**(A), **shrink**(A), **access**(i, A), which are as above.

We first look at some existing implementations. Brodnik et al. [1] gave an implementation of an EA with w -bit records that takes $n + O(\sqrt{n})$ words of space, where n is the current number of records in the array. This structure supports **access** in $O(1)$ time while supporting **grow** and **shrink** in $O(1)$ amortised time in the model $\mathcal{M}_A$ [1, p4]. Hagerup et al. [11] showed how to maintain a collection of EAs with w -bit records in $\mathcal{M}_B$, using $O(n)$ words of space where n is an *a priori* upper bound on the total size of the EAs. They supported all operations in $O(1)$ worst-case time, but assumed that the application keeps track of the starting position (which may change) of each EA. The space bound is $O(n + a)$ words where a is the number of EAs, if our interface is supported [12]. We now show how to maintain a collection of EAs with small memory overhead while supporting all the operations efficiently in both $\mathcal{M}_A$ and $\mathcal{M}_B$:

Lemma 1. *Let a be the current number of EAs in a collection of EAs and let s be the current nominal size of the collection. Then, this collection can be stored in $s + O(aw + \sqrt{sa}w)$ bits of space in $\mathcal{M}_A$, while supporting **access**, in $O(1)$ worst-case time, **create**, **grow** and **shrink** in $O(1)$ amortised time, and **destroy**(A) in amortised $O(s'/w)$ time, where s' is the current nominal size of A . In $\mathcal{M}_B$ the same holds, except that the space bound is $s + O(a^*w + \sqrt{sa^*}w)$ bits, where a^* is the maximum number of EAs that ever existed simultaneously in the past.*

Proof. We first discuss space usage in $\mathcal{M}_A$. In our implementation, an EA in the collection, whose nominal size is s' , consists of $O(\sqrt{s'/w})$ data blocks of sizes $1, 2, \dots, k$, where k is the minimum integer such that $\sum_{i=1}^k i \geq s'/w$, pointed to by a *pointer block* of size $O(\sqrt{s'/w})$. All the records of the EA are stored consecutively by considering the data blocks, in the increasing order of their sizes, as a sequence of (at least s') bits. Thus, the overhead of an EA with nominal size s' is $O(1 + \sqrt{s'/w})$ words. Now, suppose the i^{th} EA currently has

n_i records each of size r_i bits, for $1 \leq i \leq a$, and thus $s = \sum_{i=1}^a n_i r_i$. The total overhead in the EAs is $O(\sum_{i=1}^a (1 + \sqrt{n_i r_i / w}))$ words. This is maximised when all arrays have nominal size s/a , and is bounded by $O(a + \sqrt{sa/w})$ words. So the total space overhead is $O(aw + \sqrt{sa/w})$ bits as claimed.

For $\mathcal{M}_B$, we use the representation shown in Figure 1. The data in the EAs are stored in equal-sized data blocks of $k = \Theta(\sqrt{s/a^*w})$ bits each; k is changed if s or a^* doubles or halves since the last change to k . The data blocks of an EA need not be stored consecutively, and each EA has a pointer block that holds the location of its data blocks. Finally a *name* array provides the indirection needed to allow EAs to be referred to by their names. Briefly, the pointer blocks and name array are stored “loosely”, wasting a constant factor space, but this only affects the lower-order terms. $\square$

Corollary 1. *If in Lemma 1 at all times $a = \Omega(a^*)$, then the space overhead is the same for $\mathcal{M}_A$ and $\mathcal{M}_B$. If a (or a^*) is $o(s/w)$ at all times, the space used is $s + o(s)$ in $\mathcal{M}_A$ (or $\mathcal{M}_B$).*

Dynamic array. A key subroutine we use is for the *dynamic array* (DA) problem: Given a sequence of l records of $O(w)$ bits each, to support the operations of inserting a new record after, accessing a record at, and deleting a record at, a given index. In contrast to an EA, records may be inserted in the middle of the sequence, making it impossible to handle all operations in $O(1)$ time in general [8]. Hence we consider small DAs; the challenge is that we require the memory overhead to be small. We show (proof omitted):

Lemma 2. *For any constants $c, \epsilon > 0$, there is a DA that stores a sequence of length $l = w^c$, and supports accesses in $O(1)$ time and updates (insert/delete) in $O((\lg w)^{1+\epsilon})$ amortised time, using $o(l)$ bits of extra space.*

3 A Succinct Dynamic Dictionary

As mentioned above, we consider several—in fact, three—cases based on the *density* of the set, namely how close n is to m . We begin with a simple but key observation regarding the dynamic dictionary of Dietzfelbinger et al. [5] that enables us to absorb its high space usage. Their dictionary takes at most $35(1+c)n$ words of $\lg m$ bits each, for any fixed $c > 0$, where n is the current size of the set, and supports **member** in $O(1)$ time and **insert** in $O(1)$ expected amortised time. The ideas are to store the keys in an EA and store only pointers to this EA in the hash table, and to use universe reduction [7] to represent secondary hash functions more compactly. We get the following (proof omitted):

Lemma 3. *A set $S \subseteq [m]$ can be stored using two extendible arrays of total nominal size at most $(n+1) \lg m + 280n \lg n$ bits of space, while supporting **insert** in $O(1)$ expected amortised time and **member** in $O(1)$ time, where $n = |S|$. This structure requires $O(n)$ expected initialisation time.*

Sparse sets. We give an approach that works for all densities but is optimal only for sufficiently sparse sets. If $n \leq w^4$, we use the structure of Lemma 3 to represent the set. Otherwise, the data structure is a tree of constant depth, whose leaves contain sets of keys. We describe the tree in terms of a recursive construction algorithm for a static set S , and sketch the dynamisation later.

If $n > w^4$, we let N be a power of 2 such that $n \in [N, 2N)$. The root of the tree consists of a *bucket array* B of size N/w^2 , each element of which can store a w -bit pointer. We place each key in S into bucket i , $i = 0, \dots, N/w^2 - 1$ depending upon the value of its top $\lg N - 2 \lg w$ bits, and denote by S_i the set of keys placed into the i -th bucket. For $i = 0, \dots, N/w^2 - 1$, we recurse on S_i . Since all keys in S_i have the same top-order $\lg N - 2 \lg w$ bits, we omit them; thus, e.g., all keys in $S_0, \dots, S_{N/w^2-1}$ are of length $\lg m - \lg N + 2 \lg w$ bits. A pointer to the root of the representations of S_i is placed in the i -th location of the root's bucket array, for all i . If ever a recursive problem has size $\leq w^4$, the set of (shortened) keys is represented using Lemma 3. In addition, recursion is terminated perforce after λ levels where λ is a constant to be determined later, and any remaining sets are represented using Lemma 3.

Lemma 4. *The total sizes of all the bucket arrays put together is $O(n/w^2)$.*

Proof. A set T at some level of recursion is refined using a bucket array of $O(|T|/w^2)$ locations. We ‘charge’ $O(1/w^2)$ locations to each element of T . Since the depth of recursion is $O(1)$, each element is charged for $O(1/w^2)$ locations overall. $\square$

Lemma 5. *The space used by the data structure is $\mathcal{B} + O(n \lg w + \sqrt{nw})$ bits.*

Proof. If $n < w^4$ the claim follows easily: the total nominal size of the EAs used is $s = (n+1) \lg m + O(n \lg n)$ bits. Since $O(1)$ EAs are used, by Lemma 2, the space overhead in either model is $O(w + \sqrt{sw})$ bits. Since $\sqrt{s} = O(\sqrt{n \lg m} + \sqrt{\lg m} + \sqrt{n \lg n})$, $\sqrt{sw} = O(\sqrt{nw})$. Thus the total space usage is $n \lg m + O(n \lg n + \sqrt{nw}) = \mathcal{B} + O(n \lg w + \sqrt{nw})$.

Now we assume that $n \geq w^4$. We allocate the bucket arrays as EAs by means of **create** and **grow** operations. By Lemma 4 the total nominal size of these EAs is $O(n/w) = o(n)$ bits. The data structure also consists of a number of leaves where Lemma 3 is applied. Arbitrarily number the leaves $1, 2, \dots, l$, let the set at the i -th leaf be L_i , and let $n_i = |L_i|$. Note that $l = O(n/w^2)$ by Lemma 4, and that $\sum_i n_i = n$. We now add up the nominal space usage in the leaves.

Consider the i -th leaf, and suppose the set L_i stored there is obtained by refining (through bucketing) a series of sets $S \supseteq S^{(1)} \supseteq \dots \supseteq S^{(k)} \supseteq L_i$. For $j = 1, \dots, k$ let $N^{(j)}$ be a power of 2 such that $N^{(j)} \leq |S^{(j)}| < 2N^{(j)}$. Recall that the keys in $S^{(j)}$ are placed into $N^{(j)}/w^2$ buckets, and so all keys in $S^{(j+1)}$ have the same value in their $\lg N^{(j)} - 2 \lg w$ (next) most significant bits, and these bits may be omitted. Thus, the keys in L_i are of length $\lg m - \lg N - \lg N^{(1)} - \dots - \lg N^{(k)} + 2k \lg w \leq \lg m - \lg |S| - \lg |S^{(1)}| - \dots - \lg |S^{(k)}| + (2k+1) \lg w \leq \lg(m/n) - k \lg n_i + (2k+1) \lg w$.

If $|L_i| \leq w^4$ then clearly L_i is stored in EAs of total nominal size $(n_i + 1) \lg(m/n) + O(n_i(\lg n_i + \lg w)) = n_i \lg(m/n) + O(w + n_i \lg w)$ bits. Otherwise, L_i has undergone bucketing λ times, and it is stored in $(n_i + 1) \lg(m/n) + O(n_i \lg n_i) - (\lambda - 1)n_i \lg n_i + n_i(2k + 1)\lambda \lg w = n_i \lg(m/n) + O(w + n_i \lg w)$ bits if λ is large enough. Summing over all i gives that the overall nominal space usage is $n \lg(m/n) + O(n/w + n \lg w) = \mathcal{B} + O(n \lg w)$ bits.

By Corollary 1, the overhead of maintaining the EAs in both models is the same. Letting $s = \mathcal{B} + O(n \lg w) = O(nw)$ and $a = O(n/w^2)$, this overhead is $O(\sqrt{saw} + aw) = O(n + n/w) = O(n)$ bits, which is negligible. $\square$

Dynamising this data structure is straightforward. Insertions into the leaves are handled by Lemma 3. We also maintain the invariant that a bucket array x of size b has between bw^2 and $2bw^2$ keys under it. When this is violated, we rebuild the entire data structure under x , growing x to size $2b$. The amortised cost of this rebuilding at x is clearly $O(1)$, and as the depth of the tree is constant, the overall cost incurred by an insertion is also $O(1)$. Hence, we have:

Theorem 1. *A set $S \subseteq [m]$, $|S| = n$, can be represented using $\mathcal{B} + O(n \lg w + \sqrt{nw})$ bits, while supporting member in $O(1)$ worst-case time and insert in $O(1)$ expected amortised time. The space bound is valid in both $\mathcal{M}_A$ and $\mathcal{M}_B$.*

Denser Sets. Let k be a power of 2 such that $w^{\lg w} \leq 2^k < w^{2 \lg w}$. If $m/n > 2^k$, then $\mathcal{B} = \Omega(n(\lg w)^2)$, and the bound of $\mathcal{B} + O(n \lg w + \sqrt{nw})$ is easily seen to be $\mathcal{B} + o(\mathcal{B})$. Hence, we focus on the case $m/n \leq 2^k$.

Again, we proceed via bucketing. We allocate a bucket array of size $b = m/2^{2k}$ and divide S into b buckets, based on the top $\lg b = \lg m - 2k$ bits of each key. Within each bucket the keys may be truncated to their lowest-order $2k$ bits alone. The keys in bucket i are represented in an EA with $2k$ -bit records, and the name of this EA is placed in the i -th location of the bucket array. The representation of the keys in this EA is somewhat unusual. We use the EA to simulate the memory of a word RAM with word size $2k$. We say that a non-negative function f is *smooth* if $|f(t) - f(t')| = O(t' - t)$ for any integers t, t' with $t' > t \geq 0$. It is easy to show (proof omitted):

Proposition 1. *Let $m(t)$ be a smooth and $O(1)$ -time computable upper bound on the space usage (in $\mathcal{M}_B$) of a RAM algorithm. Then the algorithm can be simulated with a constant factor amortised time overhead, in an EA comprising $m(t)$ records of w' bits each, where $w' \leq w$ is the word size of the simulated RAM.*

We simulate the algorithm of Theorem 1 running in $\mathcal{M}_B$ to represent the keys in each bucket. Since this algorithm ultimately relies on Lemma 1 to manage memory, an inspection of the proof of this lemma shows that there is an accurate and easily-computable upper bound on its space usage, and that the bound is smooth. Letting $\mathcal{B}_i = \mathcal{B}(2^{2k}, n_i)$, where n_i is the number of keys in the i -th bucket, we see that the algorithm of Theorem 1 can be simulated in an EA, whose nominal size is at most $\mathcal{B}_i + O(n_i \lg k + \sqrt{n_i k})$ bits.

We now sum up these nominal sizes. Firstly, $\sum_{i=1}^b \sqrt{n_i} \leq \sqrt{bn}$; since $b = m/2^{2k} < n/2^k$, we see that $\sum_{i=1}^b k\sqrt{n_i} < kn/2^{k/2} = o(n)$. As noted in [2, Lemma 3.2], $\sum_{i=1}^b \mathcal{B}_i \leq \mathcal{B} + O(b)$. Thus, the total nominal size of all the EAs is $\mathcal{B} + O(n \lg k) = \mathcal{B} + O(n \lg \lg w)$ bits whenever $n \geq m/2^k$. Since the number of these EAs is $o(n/w)$, but their total nominal size is $\Omega(n)$ bits, the overhead of managing these is negligible. We have thus shown:

Lemma 6. *A set $S \subseteq [m]$, $|S| = n \geq m/(w^{2 \lg w})$, can be represented using $\mathcal{B} + O(n \lg \lg w)$ bits, while supporting member in $O(1)$ worst-case time and insert in $O(1)$ expected amortised time. The space bound is valid in both $\mathcal{M}_A$ and $\mathcal{M}_B$.*

Very Dense Sets. If $m/w^{\Omega(1)} > n \geq m/w^{2 \lg w}$, $\mathcal{B} = \Omega(n \lg w)$, so the space used by the data structure of Lemma 6 is $\mathcal{B} + o(\mathcal{B})$. Smaller values of n are covered by Theorem 1. We now focus on the case $n \geq m/w^\epsilon$, for positive constant $\epsilon < 1/4$ and show (proof omitted):

Lemma 7. *A set $S \subseteq [m]$, $|S| = n \geq m/w^\epsilon$, for some $0 < \epsilon < 1/4$, can be represented using $\mathcal{B} + O(m/w^{1/4})$ bits, while supporting member in $O(1)$ worst-case time and insert in $O(1)$ expected amortised time. The space bound is valid in both $\mathcal{M}_A$ and $\mathcal{M}_B$.*

The space bound of Lemma 7 is $\mathcal{B} + o(\mathcal{B})$ for $n \geq m/w^{1/5}$. For $m/(w^{2 \lg w}) \leq n < m/w^{1/5}$, the space bound of Lemma 6 is $\mathcal{B} + o(\mathcal{B})$. Smaller values of n are covered by Theorem 1. Thus we have:

Theorem 2. *A set $S \subseteq [m]$, $|S| = n$, can be represented using $\mathcal{B} + o(\mathcal{B})$ bits, while supporting member in $O(1)$ worst-case time and insert in $O(1)$ expected amortised time. The space usage may be measured either in $\mathcal{M}_A$ or $\mathcal{M}_B$.*

Satellite data. Here we describe how to augment our dynamic dictionary, so that s -bit satellite data can be associated with the keys, without excessive space overhead. We show:

Theorem 3. *A set $S \subseteq [m]$, $|S| = n$ where each element has an associated satellite data of $s \leq w$ bits, can be represented using $\mathcal{B} + ns + o(\mathcal{B} + ns)$ bits of space, while supporting member and finding the data associated with an element present in $O(1)$ time, and insert in $O(1)$ expected amortised time. The space bound is valid in both $\mathcal{M}_A$ and $\mathcal{M}_B$.*

Proof. We sketch the modifications required to the structure of Theorem 2.

case (i) $s = \omega(\lg w)$ or $n \leq m/w^{O(\lg w)}$: In this case, we represent the dictionary using the structure of Theorem 1. This consists of a constant depth tree structure whose leaves contain sets of keys which are stored using Lemma 3. This structure stores the actual keys in an EA and stores pointers to these keys in the hash table. In addition, we now store the satellite data corresponding to these keys in another EA, in the same order as their corresponding keys. Thus the pointer used

to index into the EA containing keys is also used to index into EA containing the satellite data. The overall extra space used by these EAs (containing the satellite data) is $O(n/w + \sqrt{(ns)(n/w^2)(w)}) = O(n/w + n\sqrt{s/w}) = O(n)$ bits (since $s \leq w$). Thus the overall space bound is $\mathcal{B} + ns + O(n \lg w + \sqrt{nw})$. This is $\mathcal{B} + ns + o(\mathcal{B} + n \lg s)$ whenever $s = \omega(\lg w)$ or $n = m/w^{O(\lg w)}$.

case (ii) ($s = O(\lg w)$ and $s = \omega(\lg \lg w)$) or $(m/w^{O(\lg w)} \leq n \leq m/w^{\Omega(1)})$: Here we use the solution of Lemma 6, i.e., recursively apply the above solution with word size $w' = O(\lg^2 w)$. Since $s \leq w'$ in this case, we can apply this recursively even for the satellite data. Thus a bucket containing n_i keys can be stored in an EA with nominal size $\mathcal{B}(n_i, 2k) + n_i s + O(n_i \lg w' + \sqrt{n_i w'})$. Summing this over all buckets, we get the space required in this case to be $\mathcal{B} + ns + O(n \lg k) + o(n)$. This is $\mathcal{B} + ns + o(\mathcal{B} + ns)$ whenever $s = \omega(\lg \lg w)$ or $n = m/w^{\Omega(1)}$.

case(iii) $s = O(\lg \lg w)$ and $n > m/w^{1/4}$: In this case we use the structure of Lemma 7 for very dense sets. In addition, we store the satellite data corresponding to the keys in poly-log sized blocks using a dynamic array that supports access in $O(1)$ time and updates in $O(1)$ amortized time, with a $o(1)$ bit space overhead per satellite datum. We improve upon the bounds of Lemma 2 by using precomputed tables, exploiting the fact that the record size is ‘small’. $\square$

Deletions and Further Refinements. Our approach handles deletions in the same time bound, but the details are messier. One problem is that Lemma 1 need not hold for deletions (in $\mathcal{M}_B$), as an EA with a very large name (given out when the number of EAs was large) may remain active while most lower-numbered EAs are deleted. This necessitates re-naming EAs by introducing back-pointers into the (fortunately $O(1)$) locations where the names of the EAs are held.

An issue that we have glossed over for lack of space is how to avoid excessive temporary blow-ups in space (e.g. as we move from one representation to another, or as we re-build some part of the data structure). The details vary depending on the instance. For example, when moving from the representation of Theorem 1 to the representation of Lemma 6, we build the new structure bucket by bucket, destroying one or more buckets in the old structure as each new bucket is completed. Since the buckets are of size $o(n)$ any temporary duplication is negligible. Finally, the constant factors can be greatly improved, at the expense of getting a slightly weaker result, by using [6] or [17] in place of [5].

4 Succinct Dynamic Binary Trees

We now consider the problem of representing dynamic n -node binary trees. We divide the given binary tree into connected *blocks* of size $\Theta(\lg^3 n)$ and each block into connected *subblocks* of size $\Theta(\lg n)$ (see [15]). Thus, there are $\Theta(n/\lg^3 n)$ blocks, each consisting of $\Theta(\lg^2 n)$ subblocks. Edges are classified as *intra-subblock pointers*, *inter-subblock pointers (ISPs)*, *inter-block pointers (IBPs)* and *external pointers*, in a natural way based on the endpoints of the edge. We now discuss the representations of blocks and sub-blocks. A block is represented as follows:

1. An EA A of $O(\lg^3 n)$ IBPs leaving the block, with each IBP taking $O(\lg n)$ bits. All IBPs leaving the same subblock are stored consecutively in A . Along with each IBP, we also store its pre-order number among all the IBPs leaving the block.
2. A DA B of the $\Theta(\lg^2 n)$ ISPs within this block. Each ISP is stored in $\Theta(\lg \lg n)$ bits; all ISPs leaving a subblock are stored consecutively in B .
3. A DA C of pointers to the representations (see (4)) of the $\Theta(\lg^2 n)$ subblocks of the block. These pointers are ordered so that the roots of the subblocks to which they point appear in a *pre-order consistent* manner, i.e. in the same order that they would in a pre-order traversal of the block
4. A set of $\sqrt{\lg n}$ EAs, with the i -th EA having record size $i\sqrt{\lg n}$, that store the representations of the subblocks (detailed in (8)-(11)) along with backpointers to B and C . The representation of each sub-block is padded out to the next multiple of $\sqrt{\lg n}$.
5. The prefix sums of the subtree sizes of all the child blocks of the block, where the subtree sizes appear in pre-order consistent manner.
6. The prefix sums of the sizes of all the subblocks within the block ordered in a pre-order consistent manner.
7. A constant number of $\lg n$ -bit pointers/data such as: a pointer to the parent block, position in its parent's list of IBPs (used for constant time navigation and also as backpointers), subtree size of the root node of the block, etc.

We now calculate the total nominal size of all the EAs. (1) As there are $O(n/(\lg n)^3)$ IBPs, this adds up to $O(n/\lg^2 n)$ bits over all blocks. (2, 3 or 6) $\Theta(\lg^2 n \lg \lg n)$ bits per block, or $O(n \lg \lg n / \lg n)$ bits overall. (4) We add up the sizes of the sub-block representations below. Here we only note that padding wastes $O(n/\sqrt{\lg n})$ bits overall. (5) Each prefix sum takes $O(\lg n)$ bits, but there are $O(n/(\lg n)^3)$ blocks, so this is $O(n/\lg^2 n)$ bits overall. (7) $O(\lg n)$ bits per block, or $O(n/(\lg n)^2)$ bits overall.

A sub-block is represented by concatenating the following bit-strings (we let ν be the number of nodes in this sub-block):

8. An implicit representation of the tree structure of the subblock.
9. A pointer to its list of IBPs stored in A , taking $\Theta(\lg \lg n)$ bits.
10. A pointer to its list of ISPs stored in B , taking $\Theta(\lg \lg n)$ bits. (The lengths of these lists can be obtained from the implicit representation of the subblock.)
11. The number of ISPs (within the block) before the sub-block's root in the pre-order numbering of the subblocks.
12. Number all the $\nu + 1$ edges leaving the sub-block $0, \dots, \nu$ in pre-order. Let $S, S' \subseteq [\nu + 1]$ be the set of ISPs and IBPs leaving the sub-block, respectively. A bit-string that contains $|S|, |S'|$ (using $\Theta(\lg \lg n)$ bits), followed by a bit-string of $\mathcal{B}(\nu + 1, |S|)$ bits that represents S implicitly (and likewise for S').

The size of this bitstring depends on ν , $|S|$ and $|S'|$. (8) takes 2ν bits, adding up to $2n$ bits overall and (9, 10 or 11) all take $O(\lg \lg n)$ bits, adding up to $O(n \lg \lg n / \lg n)$ bits overall. (12) takes $O(\lg \lg n) + \mathcal{B}(\nu + 1, |S|) + \mathcal{B}(\nu + 1, |S'|)$.

Using [2, Lemma 3.2] one can verify that (12) adds up to $O(n \lg \lg n / \lg n)$ bits overall. Note that (12) always takes less than 3ν bits, so we can choose the sub-block sizes small enough that the concatenation of (8)-(12) never exceeds $(\lg n)/2$ bits, allowing table lookup.

Any satellite data is stored in pre-order consistent manner in one of two DAs associated with each blocks, one each for internal and external node data. The total nominal sizes of these DAs are bn and $b(n+1)$ bits respectively.

Operations. Recall that we assume that all traversals start from the root and end at the root. We start a traversal at the root block, find its root subblock and navigate within it until we reach an edge leading out of the subblock. We then find out whether the edge is an IBP or ISP (if not, it points to an external node) and find its rank using the implicit representation of the subblock, follow the appropriate pointer to the corresponding block/subblock and traverse that.

To find the size of the subtree rooted at the current node, we find the sum of the subtree sizes of (the roots of) all the blocks leaving the current block that are within the subtree using the partial sum structure for the subtree sizes of the blocks and their pre-order numbers. We then find the sum of the sizes of all subblocks within the current block that are within the subtree using the prefix sums of the sizes of the subblocks. To these two numbers, we add the number of nodes in the current subblock that are within the subtree (which can be found in $O(1)$ time using a lookup table) to get the required subtree size.

To find the satellite datum of the current node, we first find the number of nodes before the root of the subblock (in the pre-order of the subblocks) using the prefix sums and then find the position of the node within the subblock, using a lookup table. The sum of these numbers gives the pre-order number of the node in the block, which is used to index into the satellite data block in $O(1)$ time. To find the pre-order number of the current node, we accumulate the sum of the sizes of all the blocks that are before the current block in pre-order as we traverse the tree down to the current node. To this we add the pre-order number of the node in the current block to find its pre-order number in the tree.

Many aspects of updates are easy. For example, subblocks are updated by table lookup in $O(1)$ time; the actions that need to be taken when a subblock or block gets too big are costly but infrequent and the amortised cost is easily seen to be $O(1)$. Apart from handling updates to the DAs, the only place where care is required is in updating the prefix sums. For this we use ideas from [19]. In summary:

Theorem 4. *There is a $2n+o(n)$ -bit binary tree representation that supports the following operations in $O(1)$ time: given a node, finding its parent, left child and right child, and in the course of traversing the tree, the size of the subtree rooted at the current node, the satellite datum associated with the current node (if any) and its pre-order number. We assume that traversals start and end at the root. Inserting a leaf or a node along an edge or requires $O((\lg \lg n)^{1+\epsilon})$ time. The representation can associate $b = O(w)$ -bit satellite data with external or internal nodes, or both, at an additional space cost of $bn + o(n)$ bits, or $2bn + o(n)$ bits,*

and can access the satellite datum associated with a node in $O(1)$ time. The space bounds are valid in both $\mathcal{M}_A$ and $\mathcal{M}_B$.

Acknowledgements. We thank Torben Hagerup and Venkatesh Raman for enlightening discussions.

References

1. A. Brodnik, S. Carlsson, E. D. Demaine, J. I. Munro and R. Sedgewick. Resizable arrays in optimal time and space. In *Proc. WADS '99*, LNCS 1663, 37–48.
2. A. Brodnik and J. I. Munro. Membership in constant time and almost minimum space. *SIAM J. Computing* **28** (1999), 1628–1640.
3. J. G. Cleary. Compact hash tables using bidirectional linear probing. *IEEE Trans. Comput.* **9** (1984), 828–834.
4. T. H. Cormen, C. E. Leiserson, and R. L. Rivest. *Introduction to Algorithms*, The MIT Press, Cambridge, MA, 1990.
5. M. Dietzfelbinger, A. Karlin, K. Mehlhorn, F. Meyer auf der Heide, H. Rohnert and R. E. Tarjan. Dynamic perfect hashing: upper and lower bounds. *SIAM J. Computing*, **23** (1994), 738–761.
6. D. Fotakis, R. Pagh, P. Sanders and P. Spirakis. Space efficient hash tables with worst case constant access time. In *Proc. 20th STACS*, LNCS 2607, 271–282, 2003.
7. M. L. Fredman, J. Komlós and E. Szemerédi. Storing a sparse table with $O(1)$ worst case access time. *J. ACM*, **31** (1984), 538–544.
8. M. L. Fredman and M. Saks. The cell probe complexity of dynamic data structures. In *Proc. 21st ACM STOC*, 345–354, 1989.
9. R. Grossi and J. Vitter. Compressed suffix arrays and suffix trees with applications to text indexing and string matching. *Proc. 32nd ACM STOC*, 397–406, 2000.
10. T. Hagerup. Sorting and searching on the word RAM. In *Proc. 15th STACS*, LNCS 1373, 366–398, 1998.
11. T. Hagerup, K. Mehlhorn and J. I. Munro. Maintaining discrete probability distributions optimally. In *Proc. 20th ICALP*, LNCS 700, 253–264, 1993.
12. T. Hagerup and R. Raman. An efficient quasidictionary. In *Proc. 8th SWAT*, LNCS 2368, 1–18, 2002.
13. G. Jacobson. Space efficient static trees and graphs. In *Proc. 30th IEEE Symp. FOCS*, 549–554, 1989.
14. J. I. Munro and V. Raman. Succinct representation of balanced parentheses, static trees and planar graphs. In *Proc. 37th IEEE Symp. FOCS*, 118–126, 1997.
15. J. I. Munro, V. Raman and A. Storm. Representing dynamic binary trees succinctly. In *Proc. 12th ACM-SIAM SODA*, 529–536, 2001.
16. R. Pagh. Low redundancy in static dictionaries with constant query time. *SIAM J. Comput.*, **31** (2001), 353–363.
17. R. Pagh and F. F. Rodler. Cuckoo Hashing. *Proc. 9th ESA*, LNCS 2161, 121–133, 2001.
18. R. Raman, V. Raman and S. S. Rao. Succinct indexable dictionaries, with applications to representing k -ary trees and multisets. In *Proc. 13th ACM-SIAM SODA*, 233–242, 2002.
19. R. Raman, V. Raman and S. S. Rao. Succinct dynamic data structures. In *Proc. 7th WADS*, LNCS 2125, 426–437, 2001.
20. A. Silberschatz, P. B. Galvin and G. Gagne. *Operating System Concepts*, 6th ed. John Wiley & Sons, 2001.